新闻推荐系统项目计划

Oct 6, 2026 · @Violet

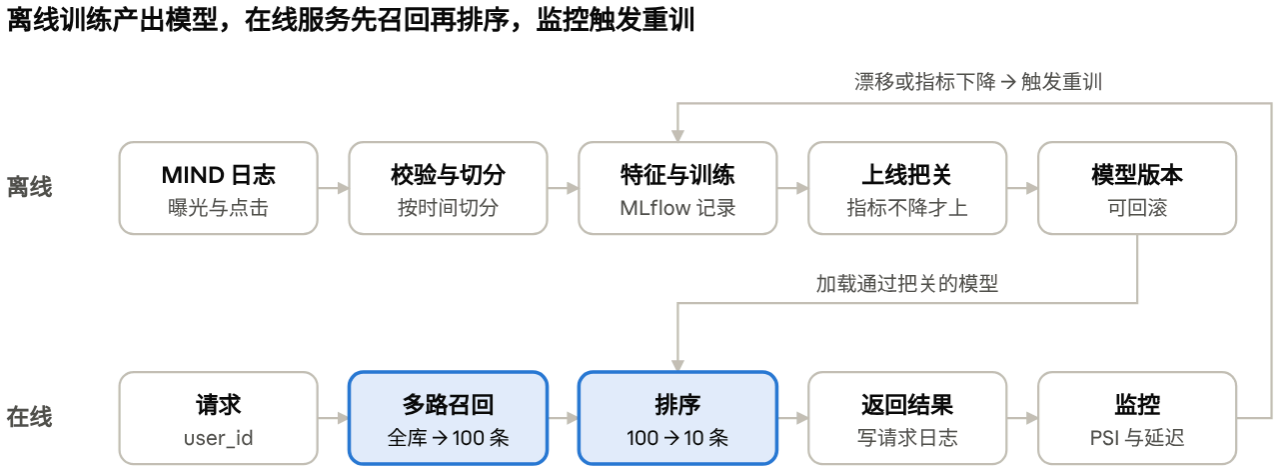
项目概览

目标：约 8 周、每周约 15 小时，在 MIND-small 新闻数据上做出一个**两阶段推荐系统**，把"训练 → 部署 → 监控"整条工业流程亲手跑一遍。策略是**先搭骨架、再逐块升级**：第 3 周末整条链路就能跑通，之后每次只替换一个模块，并记录指标变化。

最终交付物

- 一个公开的 GitHub 仓库：结构清晰，`make train` / `make serve` 一条命令就能跑
- 一个 FastAPI 推荐接口：输入 user_id，返回 10 条新闻
- 一张结果表：热门 → LR → LightGBM → Wide & Deep 各版本的 AUC、NDCG@10、Recall@K
- 监控脚本（PSI 漂移检测）和上线把关规则
- README、架构图、简历 bullet、面试讲稿

项目整体完成的标准：别人 clone 仓库后，按 README 30 分钟内能复现结果；面试时你能不看笔记讲清"数据怎么来、模型怎么选、怎么评估、上线后怎么监控"。



系统架构 · 离线 5 步，在线 5 步

上面一行是离线流程，下面一行是每次请求的在线流程；高亮的召回和排序是阶段二、三要升级的两块，其余框在骨架阶段就要先搭好。

阶段	周次	结束时应该有的东西
0 准备	第 1 周前 2 天	环境、仓库、数据都就位

阶段	周次	结束时应该有的东西
一 端到端骨架	第 1–3 周	最简单版本的完整链路跑通，有第一版指标
二 召回升级	第 4 周	item-CF + embedding/FAISS，Recall@K 对比表
三 排序升级	第 5–6 周	手写 LR、LightGBM、Wide & Deep，AUC/NDCG 对比表
四 工程化	第 7 周	schema 校验、测试、上线把关、Docker
五 收尾	第 8 周	README、架构图、简历、面试准备

时间紧时的砍法：Wide & Deep 和 Docker 可以往后放，其余都属于核心，尽量保留。

第 0 步：环境与数据准备（约 4 小时）

先把工具和数据准备好，后面只管写代码。**建议在本地电脑上写代码、用 Git 管理**；只在训练 PyTorch 模型时才用 Colab 的 GPU。前面的阶段 CPU 完全够用。

1. **建仓库**：在 GitHub 新建 `news-recsys`，加 Python 的 `.gitignore`，并把 `data/` 和 `mlruns/` 加进去（数据和实验记录不上传）。
2. **建环境**：用 conda 或 venv 建一个 Python 3.11 环境，先装 `pandas numpy scikit-learn mlflow fastapi uvicorn pyyaml pytest`。LightGBM、FAISS、PyTorch 等用到时再装。用 `pip freeze > requirements.txt` 记录依赖。
3. **下载数据**：在 MIND 官网下载 **MIND-small** 的 train 和 dev 两个压缩包，解压到 `data/raw/`。每个包里有 `behaviors.tsv`（曝光和点击日志）和 `news.tsv`（新闻类别、标题、摘要）。下载前先看一下官网的字段说明和使用协议。
4. **建目录**：按下面的结构建好空文件夹和空的 `__init__.py`，先 commit 一次。

```
news-recsys/
├── configs/config.yaml        # 所有参数：路径、特征、模型超参
├── data/raw, data/processed   # 不上传 GitHub
├── src/
│   ├── data/          # 解析日志、构造样本、切分
│   ├── features/      # 特征处理（训练和服务共用）
│   ├── retrieval/     # 召回
│   ├── ranking/       # 排序模型
│   ├── evaluation/    # 指标
│   ├── serving/       # FastAPI
│   └── monitoring/    # 日志、漂移检测
├── scripts/           # train.py、evaluate.py 等入口
├── tests/
├── notebooks/         # 只用来做探索，正式代码放 src/
├── Makefile
└── README.md
```

完成标准：`import pandas` 能读出 `behaviors.tsv` 的前 5 行；仓库里有第一个 commit。

阶段一：端到端骨架（第 1–3 周）

这一阶段只追求**跑通**，指标差没关系。每个模块都用最简单的实现，但接口要一开始就定好，后面升级时只换里面的实现。

第 1 周：读懂数据、构造样本、切分

做什么

1. **解析曝光日志**（`src/data/parse.py`）。`behaviors.tsv` 每行是一次曝光：曝光 ID、用户 ID、时间、用户历史点击（空格分隔的新闻 ID）、本次展示的候选列表（如 `N123-1 N456-0`，-1 是点了，-0 是没点）。把候选列表**展开成一行一个候选**：`impression_id, user_id, time, news_id, label`，存成 parquet。
2. **数据探索**（`notebooks/01_eda.ipynb`）。先回答这几个问题，并把答案记下来（以后写 README 和面试都会用到）：
   - 整体点击率是多少？正负样本比例多悬殊？
   - 每次曝光平均有几个候选？用户历史长度的分布如何？有多少用户历史为空（冷启动）？
   - 时间范围是哪几天？每天的曝光量和点击率有什么变化？
   - 哪些新闻类别最多？哪些类别点击率最高？
3. **按时间切分**（`src/data/split.py`）。MIND-small 的 train 和 dev 本身就是按时间先后分的：用 train 的最后一天做验证集，前面的做训练集，官方 dev 当测试集。**不要随机切分**，否则会用未来的数据预测过去，就是数据泄露。

4. **热门召回 baseline**（`src/retrieval/popular.py`）。统计每条新闻在训练集里的点击数，对每个用户返回点击数最高、且用户没看过的 Top-100。写成一个类，有 `fit(train_df)` 和 `retrieve(user_id, k)` 两个方法，后面所有召回器都用同一套接口。

完成标准：有一份带答案的数据探索笔记；`train / val / test` 三个 parquet 文件；热门召回能对任意用户返回 100 条新闻。

这周学到的：曝光日志怎么变成训练样本、为什么要按时间切分、类别不平衡、冷启动。

第 2 周：第一个排序模型和评估

做什么

1. **写 3 个简单特征**（`src/features/build.py`）：新闻的历史点击率、用户历史长度、候选新闻的类别在用户历史里占的比例。写成一个函数 `build_features(user, candidates, stats)`，**训练和服务都调用它**。新闻点击率这类统计量只能用训练集算，算好后存成 `stats` 文件。
2. **训练 sklearn 的 LogisticRegression**（`src/ranking/lr.py`）。和召回器一样，统一用 `fit` 和 `predict_proba` 两个方法。
3. **自己写评估指标**（`src/evaluation/metrics.py`）。分两类，要分清楚它们评的不是一回事：
   - **排序指标**：在每次曝光的候选列表内部排序，算 AUC、MRR、NDCG@5、NDCG@10，再对所有曝光取平均。这是 MIND 的官方评测方式。AUC 可以调 sklearn，**NDCG 和 MRR 自己写**，面试常考。
   - **召回指标**：用户真正点击的新闻有多少比例出现在召回的 Top-K 里（Recall@50、Recall@100）。
4. **接入 MLflow**（`scripts/train.py`）。每次训练自动记录配置参数和指标，用 `mlflow ui` 能看到历次实验对比。

完成标准：运行 `python scripts/train.py` 一条命令，从 parquet 文件到 MLflow 里出现一行指标；另外用一个"随机打分"当对照，确认 LR 的 AUC 明显高于 0.5。

这周学到的：特征泄露、为什么要有 baseline、排序指标和召回指标的区别、实验可复现。

第 3 周：上线、监控、测试

做什么

1. **FastAPI 服务**（`src/serving/app.py`）。一个接口 `GET /recommend?user_id=U123&k=10`，内部流程：召回 100 条 → 调用 `build_features` → LR 打分 → 取前 10 条返回。模型和 `stats` 在服务启动时读入一次。未知用户直接返回热门新闻。
2. **请求日志**（`src/monitoring/logger.py`）。每次请求写一行 JSON 到 `logs/requests.jsonl`：时间、user_id、返回的新闻、每条的预测分数、耗时。

3. **PSI 漂移检测**（`src/monitoring/drift.py`）。比较训练集和日志里某个特征（或预测分数）的分布：分成 10 个桶，按公式算 PSI，超过 0.2 就打印警告。用 dev 集模拟"上线后的新数据"。
4. **两个测试**（`tests/`）：一个检查验证集的最早时间晚于训练集的最晚时间（防泄露）；一个用手算的小例子检查你的 NDCG 函数。
5. **Makefile**：`make data`、`make train`、`make serve`、`make test`。

PSI 的公式（p_i 和 q_i 分别是训练数据和新数据落在第 i 个桶的比例）：

$$\mathrm{PSI} = \sum_{i=1}^{10} \left(q_i - p_i\right) \ln \frac{q_i}{p_i}$$

完成标准（骨架完成）：从零开始依次运行 `make data && make train && make test` 全部成功；`make serve` 后在浏览器打开 `/docs` 能调用接口拿到 10 条新闻；日志文件里有记录；PSI 脚本能输出一个数字。把这时的指标当作 **v0**，记进结果表。

这周学到的：模型怎么变成服务、training-serving skew、为什么要记日志、漂移怎么量化。

阶段二：召回升级（第 4 周）

这周把热门召回升级成**多路召回**，并用 Recall@K 证明每一路都有贡献。新闻场景有一个关键特点：**每天都有大量新文章，它们还没有任何点击**。只靠点击数据的方法永远召回不到它们，所以需要用文本内容做召回。

做什么

1. **item-CF 召回**（`src/retrieval/itemcf.py`）。统计"点过 A 的用户也点过 B"的共现次数，算新闻之间的相似度（用余弦相似度归一化，避免热门新闻和所有东西都相似）。对用户历史里的每条新闻取最相似的若干条，合并打分。
2. **内容 embedding 召回**（`src/retrieval/embedding.py`）。
   - 用 sentence-transformers 里的一个小模型（如 all-MiniLM-L6-v2）把每条新闻的标题和摘要编码成向量，存成文件，只算一次。
   - 用户向量 = 历史点击新闻向量的平均。
   - 把所有新闻向量放进 **FAISS** 索引（先用最简单的 `IndexFlatIP`，向量先归一化，内积就是余弦相似度），用用户向量查最近的 Top-K。
3. **多路合并**（`src/retrieval/merge.py`）。热门、item-CF、embedding 各取一部分（比例写在配置里），去重、去掉用户看过的，凑满 100 条交给排序。服务里只需要换掉召回器，其他代码不动。
4. **对比实验**。对每种召回方式分别算 Recall@50 和 Recall@100，再**单独看"新文章"的召回率**（测试集里那些训练集中从未出现的新闻）。你大概率会看到：item-CF 对新文章完全无效，embedding 可以。这就是面试里很好讲的一个发现。

结果表模板（填你自己跑出来的数）：

召回方式	Recall@50	Recall@100	新文章 Recall@100
热门			
item-CF			
embedding + FAISS			
多路合并			

完成标准：上表填满；多路合并的 Recall@100 高于任何单一一路；服务已切换到多路召回且仍能正常返回。

这周学到的：协同过滤、物品冷启动、文本 embedding、近似最近邻检索、为什么工业界用多路召回。

可选进阶：时间充裕时，用 PyTorch 训练一个 two-tower 模型替换"平均向量"的用户表示，用 in-batch negatives 做负采样。没时间就放到最后一节的进阶清单里。

阶段三：排序升级（第 5–6 周）

这两周是 ML 基础最集中的部分：从手写 LR 一路升级到神经网络，每一步都要跟上一版比指标。**这部分代码一定自己写**，不要让 AI 代写。

第 5 周：手写 LR + 特征工程 + LightGBM

1. **numpy 手写 LR**（`src/ranking/lr_numpy.py`）。先在纸上推一遍带 L2 正则的损失和梯度，再写代码：sigmoid（注意数值溢出）、log loss、mini-batch 梯度下降。写完做两个检查：用数值微分做梯度检验；在同样的特征上，结果要和 sklearn 版本的 AUC 差不多。
2. **特征工程**（扩展 `build_features`）。每加一组特征就跑一次实验，记录它的贡献：
   - 新闻侧：类别、子类别、**新闻年龄**（发布到曝光的间隔，新闻很看时效）、近几小时的点击数
   - 用户侧：历史长度、各类别的历史点击占比
   - 交叉特征：用户对该类别的偏好、**用户向量和候选新闻向量的相似度**（直接用阶段二的 embedding）
   - 注意：所有"近 N 小时"的统计量，只能用曝光时刻**之前**的数据。
3. **LightGBM**（`src/ranking/gbdt.py`）。先用 `binary` 目标训一版，再试 `lambdarank`（直接优化排序，需要按曝光分组）。用验证集做 early stopping，调 3–4 个关键超参（`num_leaves`、`learning_rate`、`min_data_in_leaf`、`feature_fraction`）。看特征重要性，确认和你的直觉一致。

手写 LR 的损失和梯度（n 个样本，σ 是 sigmoid，λ 是 L2 系数）：

$$L(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln \hat{y}_i + (1 - y_i)\ln(1 - \hat{y}_i)\Big] + \frac{\lambda}{2}\lVert \mathbf{w} \rVert^2, \qquad \hat{y}_i = \sigma(\mathbf{w}^{\top}\mathbf{x}_i + b)$$

$$\nabla_{\mathbf{w}} L = \frac{1}{n} X^{\top}(\hat{\mathbf{y}} - \mathbf{y}) + \lambda \mathbf{w}, \qquad \frac{\partial L}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

第 6 周：Wide & Deep + 误差分析

1. **PyTorch Wide & Deep**（`src/ranking/wide_deep.py`，在 Colab GPU 上训练）。
   - Wide 部分：稀疏的交叉特征，负责"记住"规律
   - Deep 部分：类别、子类别等做 embedding，和数值特征拼接后过 2–3 层 MLP，负责"泛化"
   - 自己写 training loop：DataLoader、BCE loss、Adam、每个 epoch 算验证集 AUC、early stopping、保存最好的模型
2. **误差分析**（`notebooks/02_error_analysis.ipynb`）。对最好的模型，按分组看指标：用户历史长度（0、1–5、6–20、20+）、新文章 vs 旧文章、各新闻类别。找出模型最差的一组，写一句你认为的原因和下一步改进思路。
3. **校准检查**。画校准曲线（预测点击率 vs 实际点击率），算一下平均预测值和真实点击率的比值。广告场景里预测值会直接用来算钱，所以校准很重要。
4. **选出上线模型**。综合指标、推理速度、复杂度，决定服务里用哪个模型，并把理由写进 README。如果 LightGBM 和 Wide & Deep 效果差不多，选更简单更快的那个也完全合理，而且是很好的面试素材。

排序结果表模板：

版本	AUC	MRR	NDCG@5	NDCG@10	单次推理耗时 (ms)
随机打分					
v0 sklearn LR（3 个特征）					
numpy LR（全部特征）					
LightGBM binary					
LightGBM lambdarank					
Wide & Deep					

完成标准：上表填满；手写 LR 通过梯度检验；有一页误差分析结论；服务已切换到选定的模型。

这两周学到的：梯度推导与正则化、特征工程、boosting、pointwise 与 listwise 排序、embedding 和神经网络训练、过拟合、校准、效果和延迟的取舍。

阶段四：工程化与上线保障（第 7 周）

这周把"能跑"变成"可信"：数据不对会报错，模型不够好上不了线，换一台电脑也能跑。大部分学生项目都缺这一块，它也最能体现工业界思维。

1. **配置集中管理**。把代码里所有写死的路径、特征列表、超参、召回比例、把关阈值都搬进 `config.yaml`。固定 numpy、PyTorch、LightGBM 的随机种子。确认同一份配置跑两次，指标完全一致。
2. **数据校验**（`src/data/validate.py`）。用 Pandera 给展开后的训练数据写 schema：字段类型、label 只能是 0 或 1、关键字段无缺失、时间在合理范围内、点击率在一个合理区间内。训练前先跑校验，不通过就停止。故意穿坏一份数据，确认它真的会报错。
3. **补全测试**。在骨架的 2 个测试基础上再加几个：
   - 特征函数对同一输入输出稳定；对未知用户、空历史不报错
   - 时间相关的统计特征没有用到曝光时刻之后的数据
   - 服务接口返回恰好 k 条、不重复、不包含用户看过的新闻（用 FastAPI 的 TestClient）
   - 用同一条样本分别走"离线训练的特征路径"和"在线服务的特征路径"，结果必须完全相同（防 training-serving skew）
4. **上线把关**（`scripts/promote.py`）。新训练的模型只有同时满足以下条件，才替换当前线上模型：AUC 和 NDCG@10 不低于当前模型、校准比值在设定范围内、推理耗时不超过上限。用 MLflow Model Registry 或简单的 `models/production/` 目录管理版本，并能一条命令回滚到上一版。
5. **监控升级**。PSI 脚本扩展到多个关键特征和预测分数；从请求日志统计 p50/p95 延迟和未知用户占比；输出一份简单的监控报告，哪一项超过阈值就标红并建议重训。
6. **Docker**。写一个 Dockerfile 把服务打包，`docker run` 后接口可用。
7. **CI（可选）**。加一个 GitHub Actions 工作流，每次 push 自动跑 `pytest`。只有十几行 YAML，但很加分。

完成标准：故意放一份坏数据，训练会拒绝运行；故意训一个差模型，把关会拒绝上线；`pytest` 全绿；`docker run` 能起服务；监控报告能对 dev 集给出 PSI 和延迟。

这周学到的：可复现性、数据质量、测试、模型版本管理与回滚、监控指标怎么选、容器化。

阶段五：收尾与面试准备（第 8 周）

项目做完不等于能讲好。这周把项目包装成招聘方 3 分钟看懂、面试官 30 分钟问不倒的样子。

1. README，按这个顺序写：

1. 一句话介绍 + 架构图（召回 → 排序 → 服务 → 日志 → 监控）
2. 结果表：召回表和排序表，加一两句最重要的发现
3. 快速开始：从下载数据到调用接口的命令

4. 设计决策：为什么按时间切分、为什么多路召回、为什么选这个排序模型、把关规则怎么定的
5. 局限和下一步：离线指标不等于线上效果、曝光偏差、没做的进阶项

2. 更新简历。用之前写好的三条 bullet，把 X、Y 换成结果表里的真实数字；技术栈只写真正用到的。

3. 准备 2 分钟项目介绍（英文）。结构：做了什么 → 系统怎么分层 → 最大的一个发现（比如新文章召回）→ 结果数字 → 如果继续做会做什么。录音听一遍，压到 2 分钟以内。

4. 对着这份问题清单练习回答，每题都要能用自己项目里的例子来答：

- [ ] 数据怎么来？正负样本怎么定义？只有被展示过的新闻才有标签，这会带来什么偏差？
- [ ] 为什么不能随机切分？你在哪些地方防了数据泄露？
- [ ] 为什么要分召回和排序两个阶段？两者对模型的要求有什么不同？
- [ ] 新文章没有点击，你怎么把它们推给用户？新用户呢？
- [ ] AUC 和 NDCG 分别衡量什么？什么时候两者结论会不一致？
- [ ] 推导 LR 的梯度。L1 和 L2 正则有什么区别？
- [ ] LightGBM 和 Wide & Deep 你最后选了哪个？为什么？
- [ ] 什么是 training-serving skew？你怎么防止的？
- [ ] 模型上线后效果变差，你怎么发现、怎么定位、怎么处理？
- [ ] 离线指标提升了，线上就一定会变好吗？你会怎么设计 A/B 测试？
- [ ] 如果用户量和新闻量都增长 100 倍，系统哪里会先撑不住？
- [ ] 这个项目里你踩过最大的坑是什么？（行为面试常问，平时就把坑记下来）

5. 找人看一眼。拿着 README 请在大厂做推荐的人花 15 分钟提意见，重点问"你们实际怎么做监控和上线"。

完成标准：一个没见过这个项目的同学，按 README 能在 30 分钟内复现结果；你能不看笔记回答上面每一题。

常见坑、学习资源与进阶项

最容易踩的坑

- **数据泄露**：用全量数据算新闻点击率再当特征，指标会好得不正常。只要指标突然大幅提升，先怀疑泄露。
- **第 1 周超时**：解析和展开曝光日志常常比预想多花一倍时间。先用 1% 的数据把代码跑通，再跑全量。
- **训练和服务的特征不一致**：最常见的上线 bug。永远只保留一份特征代码。
- **一直调参**：每个阶段达到完成标准就往下走。这个项目的价值在完整流程，不在把 NDCG 再提 0.5%。

- **notebook 里的代码没搬进 src/**：探索可以在 notebook，但只要一段逻辑要被重复用，就立刻搬成函数。
- **内存不够**：展开后的样本行数会很多。用 parquet、把 ID 转成整数、用 `category` 类型，必要时对负样本下采样（下采样后要记得校正预测值）。
- **每周留记录**：在仓库里建一个 `JOURNAL.md`，每周记下做了什么、指标变化、踩了什么坑。写 README 和准备面试时会省很多事。

学习资源（按用到的顺序）

资源	什么时候看	看什么
MIND 数据集论文（Wu et al., ACL 2020）	第 1 周	数据怎么收集、官方评测指标
Andrew Ng，Machine Learning Specialization	第 2–5 周，哪里不懂补哪里	LR、正则化、评估、推荐系统那一课
Deep Neural Networks for YouTube Recommendations（Covington et al., 2016）	第 4 周	工业界为什么分召回和排序
Wide & Deep Learning for Recommender Systems（Cheng et al., 2016）	第 6 周	模型结构和"记忆 vs 泛化"
Chip Huyen，Designing Machine Learning Systems	第 7–8 周	数据分布变化、监控、部署那几章

这些都是我凭印象列的，没有附链接，按标题搜索即可。

之后可加的进阶项（主线完成后，按对面试的帮助排序）

- [ ] two-tower 召回模型（PyTorch，in-batch negatives）
- [ ] 离线 A/B replay：用历史曝光模拟新旧模型对比，做显著性检验
- [ ] 重排层：类别多样性、同一主题去重
- [ ] Redis 缓存用户特征和召回结果，对比延迟
- [ ] 用 KuaiRand 重跑一遍，验证框架能换数据集复用